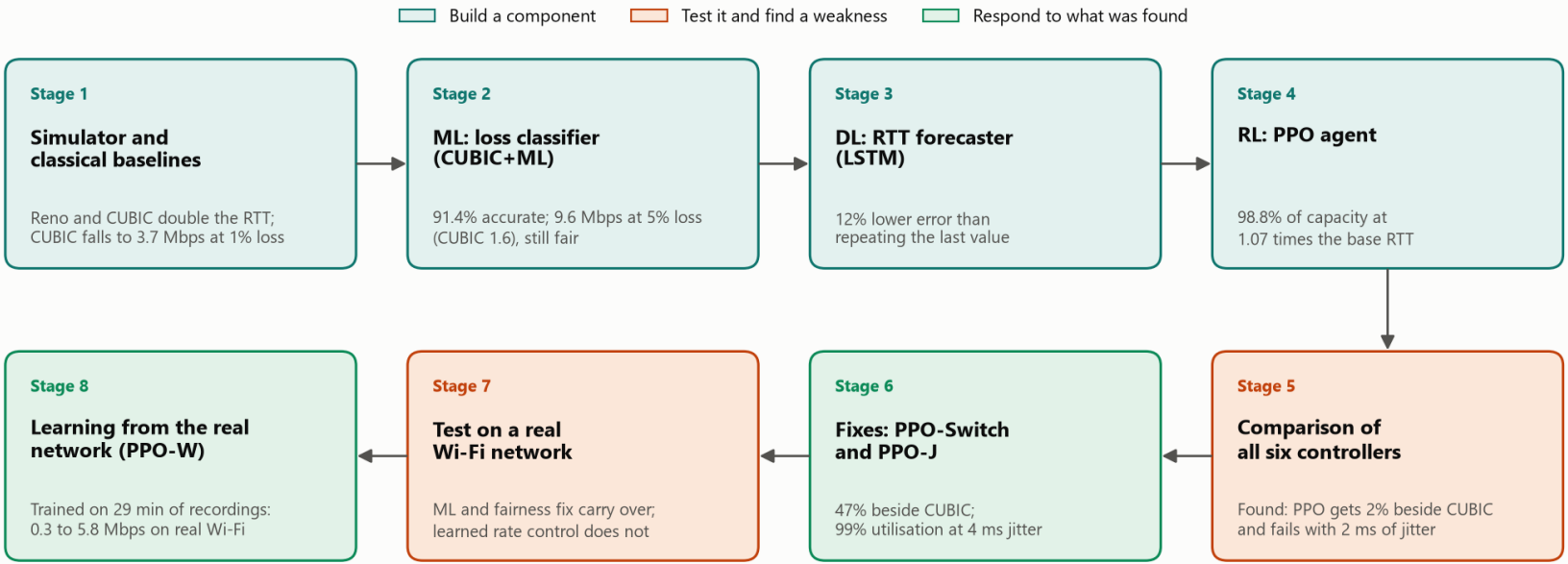
Congestion Control using ML/DL

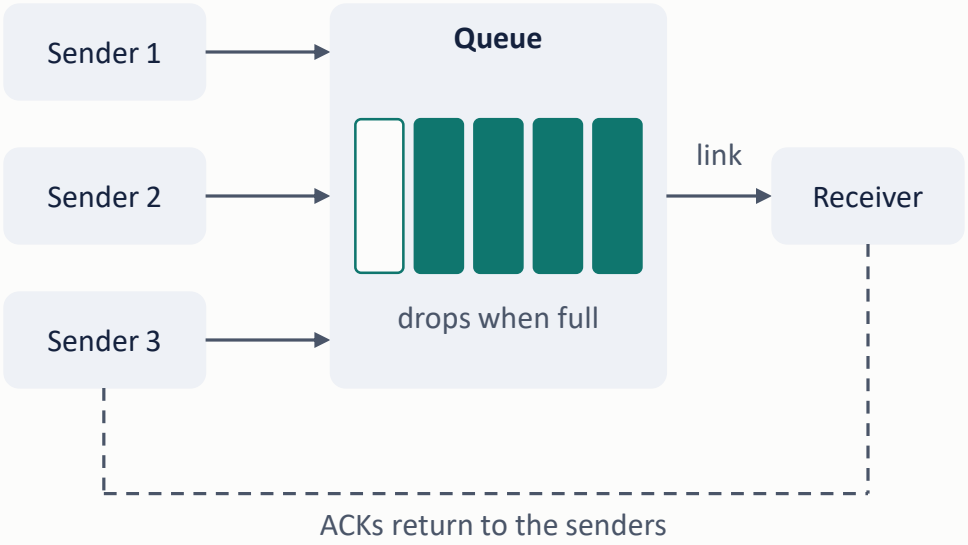
From a simulator to a real Wi-Fi network, in eight stages

Nirupam Raut (24BAI0345) · Aaditya Yadav (24BDS0450) · Computer Networks, Slot F2 · October 2026

How the project progressed



What congestion control does



Too slow

Link capacity is wasted.

Too fast

The queue fills, delay rises and packets are dropped.

The job

Find the highest rate that keeps the queue short, using only ACKs.

INTRODUCTION

Fixed rules fail in two ways

Every loss is treated as congestion

A noisy wireless link drops packets for reasons that have nothing to do with congestion. The sender slows down anyway.

10 → 3.7 Mbps

CUBIC at only 1% random loss

The queue is kept full

Reno and CUBIC back off only after the queue overflows, so every packet waits in a nearly full queue.

2×

the round-trip time of an empty queue

The question: can learned components fix both?

Goal and objectives

GOAL

Test whether machine learning, deep learning and reinforcement learning can fix both weaknesses, and report honestly where they cannot.

1. Build a packet-level simulator and three baselines: Reno, CUBIC and BBR
2. ML: classify each loss as congestion or random, and use it inside CUBIC
3. DL: forecast round-trip time with an LSTM
4. DRL: train a PPO agent to set the sending rate, plus a CUBIC+PPO hybrid
5. Compare all six on throughput, delay, loss and fairness
6. Fix the weaknesses found, and test everything on a real Wi-Fi network

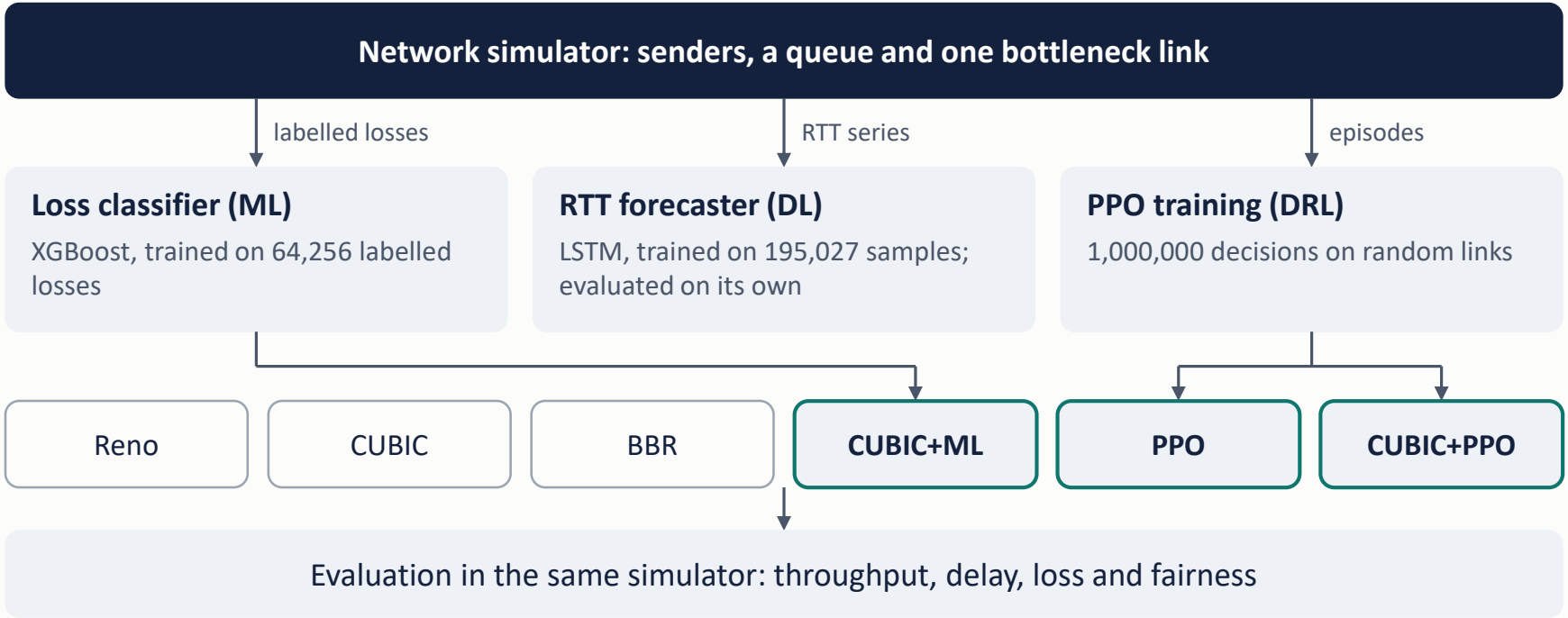
STAGE 1 OF 8

Simulator and baselines

WHY THIS STAGE

To compare controllers fairly we need one network that every controller sees in the same way, and classical baselines to beat.

How the project fits together



The network simulator

1 ms
time step, with exact ACK timestamps

1500 B
fixed packet size

Drop-tail
queue, plus optional random loss

Tagged
every loss carries its true cause

Checked against theory: Reno at 1% loss gives 3.9 Mbps; the Mathis formula predicts 3.7 Mbps.

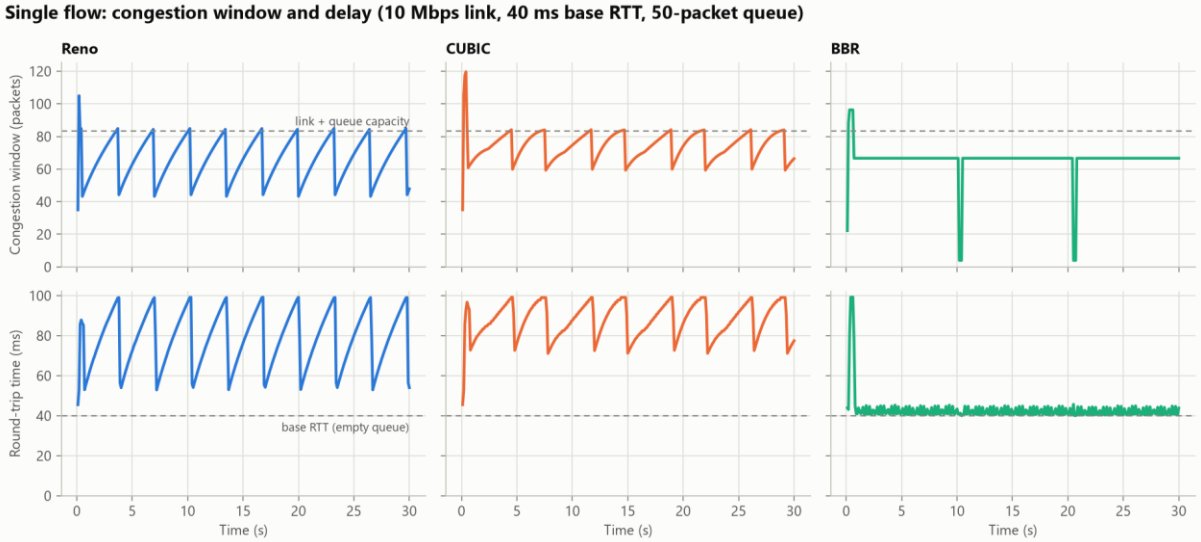
One interface for every controller

Member	Purpose
cwnd	Window, in packets
pacing_rate	Packets per second
on_ack(ack)	Called for every ACK
on_loss(loss)	Called for every loss
on_tick(now)	Optional timer

Six controllers compared

Controller	Type	How it decides
Reno	Classical	Adds 1 packet per RTT; halves the window on loss
CUBIC	Classical	Cubic window growth; × 0.7 on loss (Linux default)
BBR	Classical	Paces at the estimated link rate; ignores loss
CUBIC+ML	Machine learning	CUBIC that ignores losses a classifier judges random
PPO	Reinforcement learning	A neural network raises or lowers the sending rate
CUBIC+PPO	Hybrid	CUBIC with a neural network scaling its window

Baselines match theory



One flow on a 10 Mbps link, 40 ms base RTT, 50-packet queue. Top: congestion window. Bottom: round-trip time.

79–88 ms

average RTT with Reno and CUBIC on a 40 ms path

42 ms

average RTT with BBR

3.9 Mbps

Reno at 1% loss; theory predicts 3.7

STAGE 2 OF 8

Machine learning

WHY THIS STAGE

Stage 1 confirmed that CUBIC falls from 10 to 3.7 Mbps at 1% random loss. Can a classifier tell random loss from congestion?

Data for the loss classifier

Setting	Range across the 400 scenarios
Bandwidth	1 to 30 Mbps
One-way delay	5 to 80 ms
Queue size	0.1 to 3 pipe sizes
Random loss	0 to 3%
Changing capacity	In 40% of scenarios
Flows	1 to 3 (Reno, CUBIC or BBR)

400

simulated scenarios, 30 s each

64,256

labelled losses

49,092 / 15,164

congestion / random losses

297 / 100

scenarios for training / testing

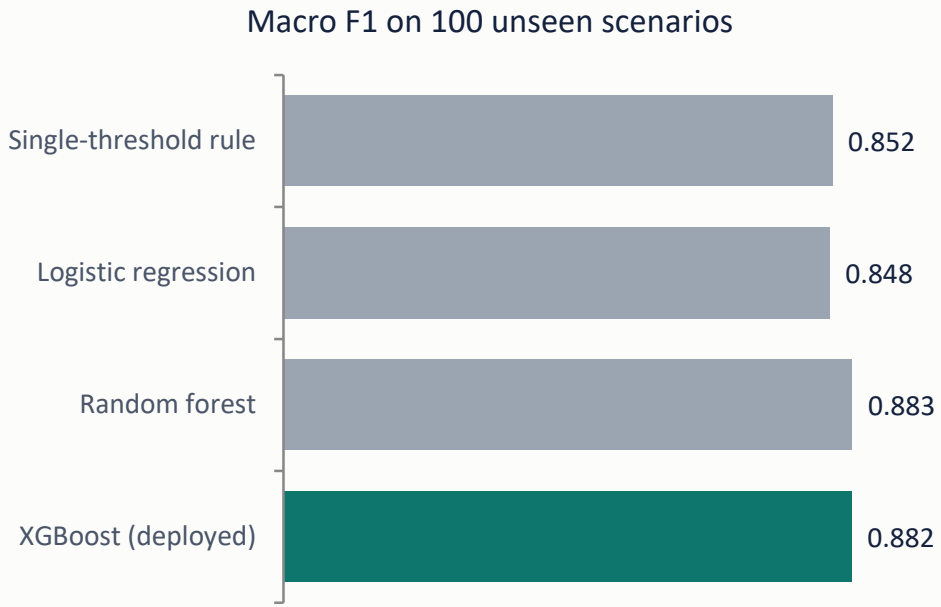
Whole scenarios are held out, so the test measures networks the classifier has never seen.

Ten signals describe a loss

rtt_ratio	Latest RTT ÷ minimum RTT	**srtt_ratio**	Smoothed RTT ÷ minimum RTT
rtt_peak_ratio	Latest RTT ÷ recent peak RTT	**rtt_trend**	RTT change over 16 ACKs
rtt_dev	RTT variation ÷ minimum RTT	**rate_ratio**	Delivery rate ÷ recent peak
inflight_ratio	Packets in flight ÷ pipe size	**losses_last_rtt**	Other losses in the last RTT
rtts_since_loss	RTTs since the previous loss	**loss_rate**	Loss rate, last ~100 packets

Intuition: congestion loss happens when the queue is full, so the RTT is near its peak. Random loss can come at any RTT.

Four classifiers compared

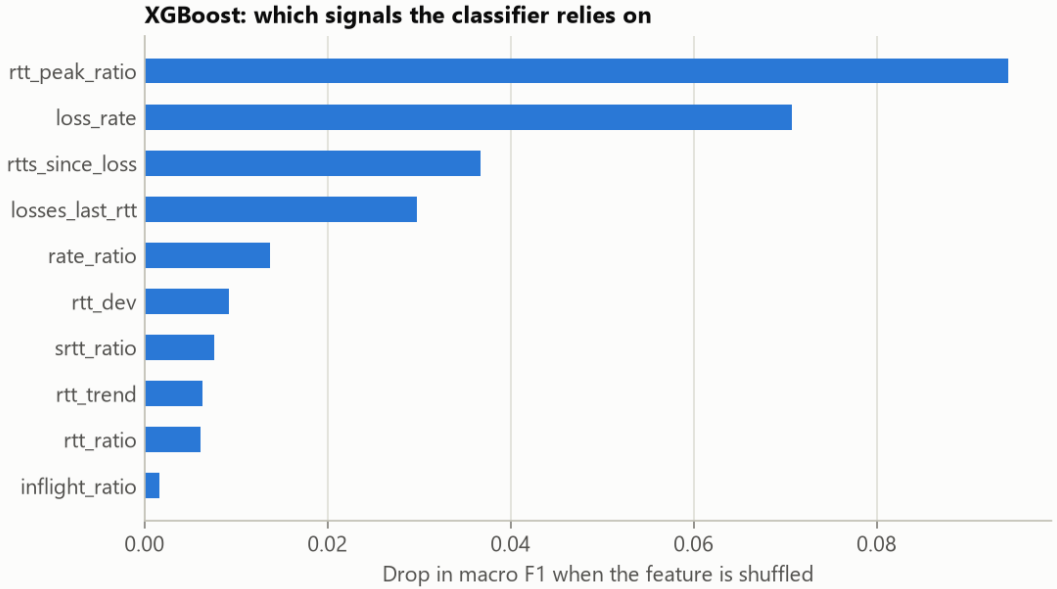


91.4%

of losses classified correctly by XGBoost on unseen networks

- Tree ensembles beat the best single threshold by 3 points of F1
- XGBoost is as accurate as the forest and much faster per loss
- Misses 5.4% of congestion losses; misreads 18.6% of random ones

What the classifier relies on



1. **RTT relative to its recent peak**
2. **Recent loss rate**
3. **Time since the previous loss**

A random loss looks different because it arrives when the queue is not full.

CUBIC+ML: using the classifier



Why 80% sure, not 50%

At 50% it ignored some real congestion losses and took 73% of a link from a normal CUBIC flow. At 80% the two share the link evenly.

9.6 Mbps

at 5% random loss; CUBIC gets 1.6

50.5%

of a link beside a normal CUBIC flow

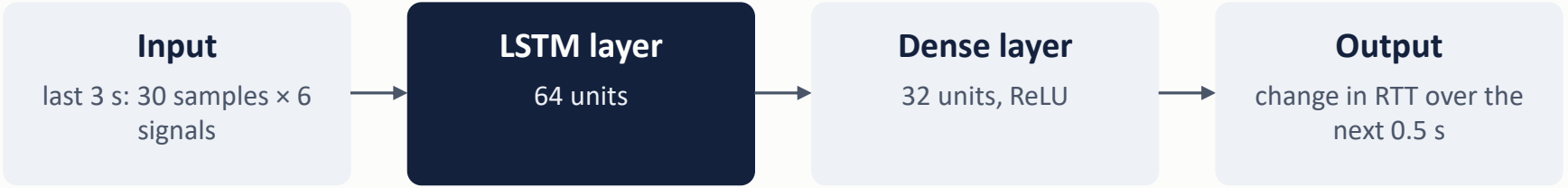
STAGE 3 OF 8

Deep learning

WHY THIS STAGE

CUBIC+ML fixed random loss but still fills the queue. Can a neural network see the delay coming?

LSTM forecaster design



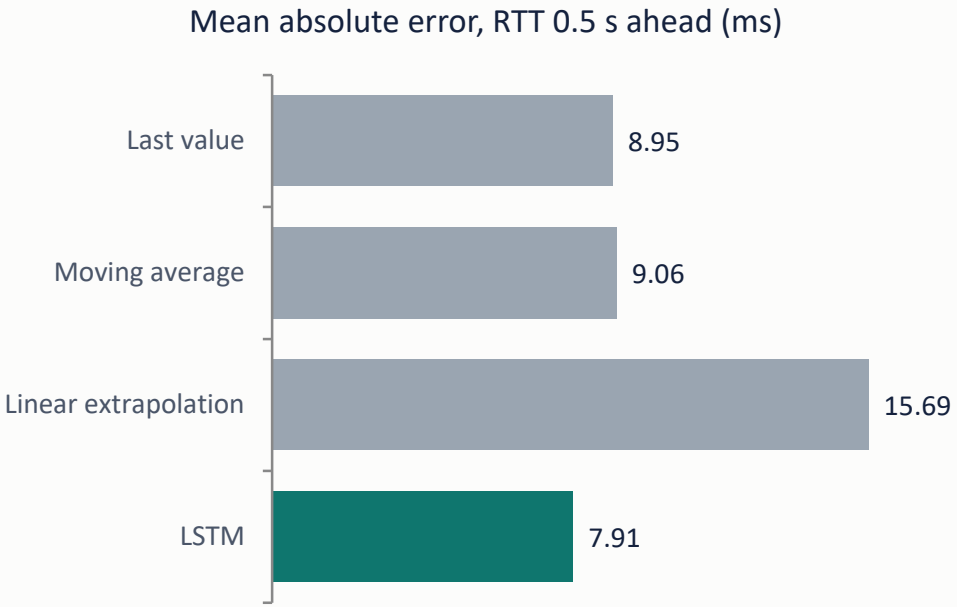
Trained on

- 195,027 samples from the same 400 scenarios
- Split by scenario: 70% train, 15% validation, 15% test
- Inputs use past measurements only

How it was trained

- Adam, learning rate 0.001, batch size 512
- Huber loss on the change in RTT
- Stops after 4 epochs without improvement
- About 20,500 parameters in total

LSTM forecast results



12%

lower error than repeating the last value

21%

lower root mean square error: it avoids the largest misses

A modest gain. It still lags sudden jumps, and no controller uses the forecast yet.

STAGE 4 OF 8

Reinforcement learning

WHY THIS STAGE

Forecasting alone gained only 12%. Can an agent learn to keep the link full and the queue short?

State, action and reward

STATE

What it sees

Four signals for each of its last 8 intervals: delay trend, delay level, send-to-ACK ratio and loss rate. 32 numbers in all.

ACTION

What it does

One number a between −1 and 1. New rate = old rate × e^(0.3a): up to +35% or −26%, once every 1.5 RTTs.

REWARD

What it wants

Throughput − 0.5 × queueing delay − 5 × congestion loss, each normalised per link.

- Every reward term is divided by the link's own capacity or base RTT, so one policy works on slow and fast links
- Random loss is not penalised: the sender did not cause it

How the agent was trained

Setting	Range in each training episode
Bandwidth	2 to 40 Mbps
Base RTT	10 to 150 ms
Queue size	0.2 to 3 pipe sizes
Random loss	None in half, else up to 3%
Capacity	Changes in half the episodes
Starting rate	5% to 150% of capacity

8 environments each run 1,024 decisions

↓

PPO scores each action against what it expected

↓

Policy network updated: 10 passes, batches of 256

Repeated about 120 times

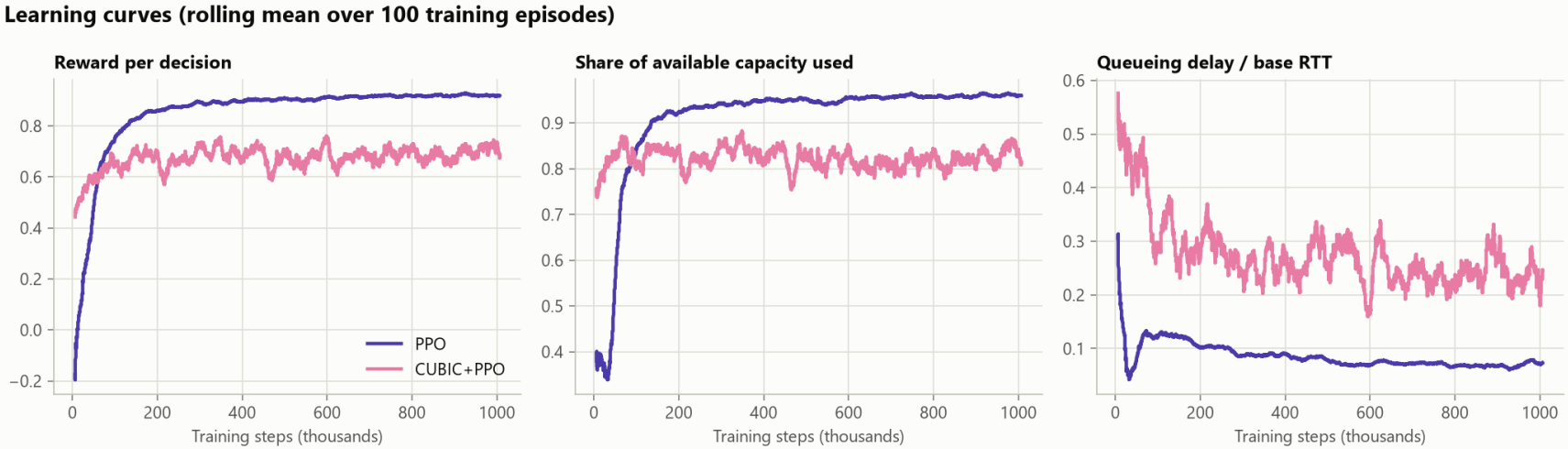
1,000,000

decisions for the main agent

≈ 20 min

on a laptop CPU, no GPU

Learning curves



PPO passes 90% of link capacity after about 200,000 decisions and keeps lowering delay. The hybrid's curve is lower partly because 30% of its episodes include a competing flow.

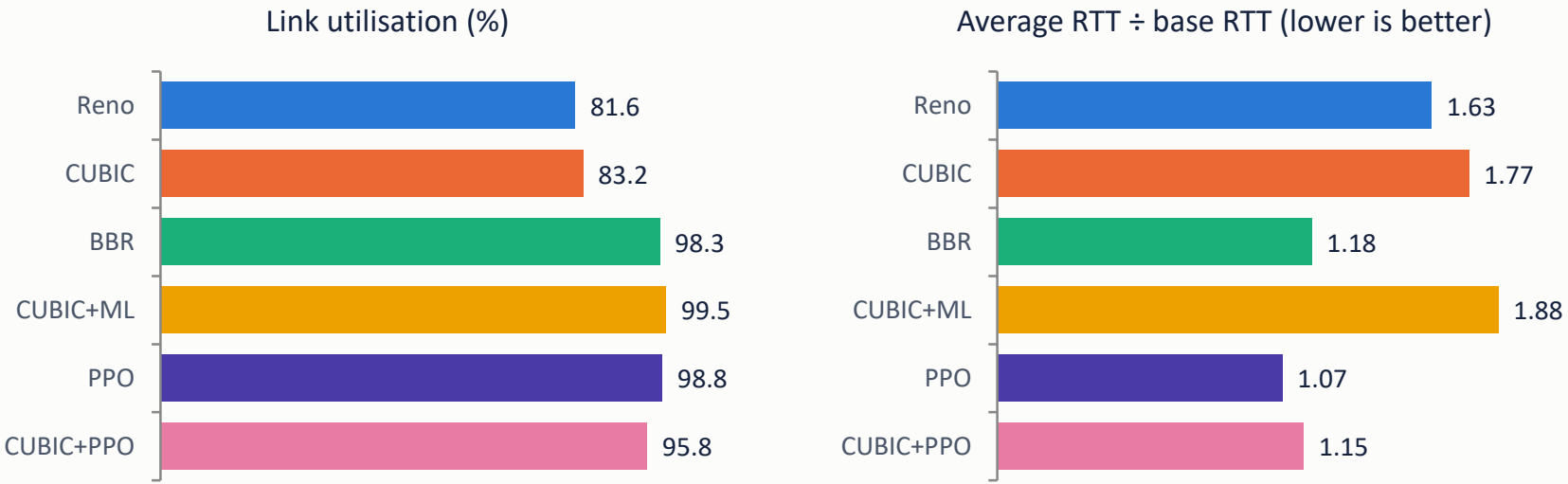
STAGE 5 OF 8

Comparison

WHY THIS STAGE

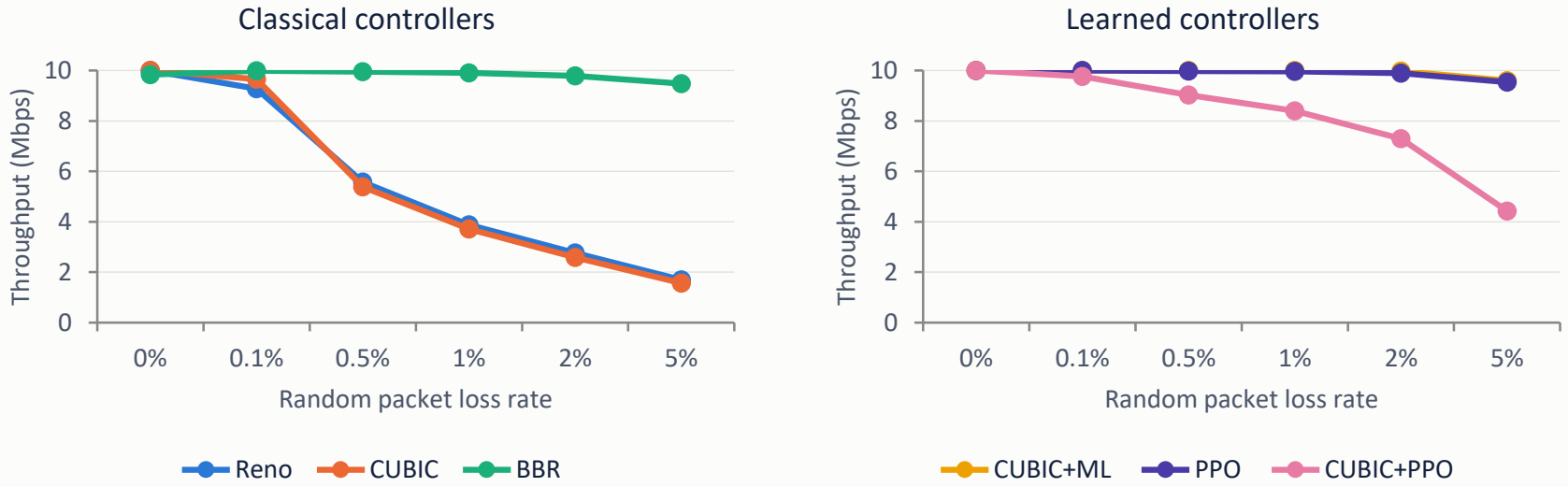
Four learned and classical controllers are ready. How do all six compare on the same links?

Results: unseen links



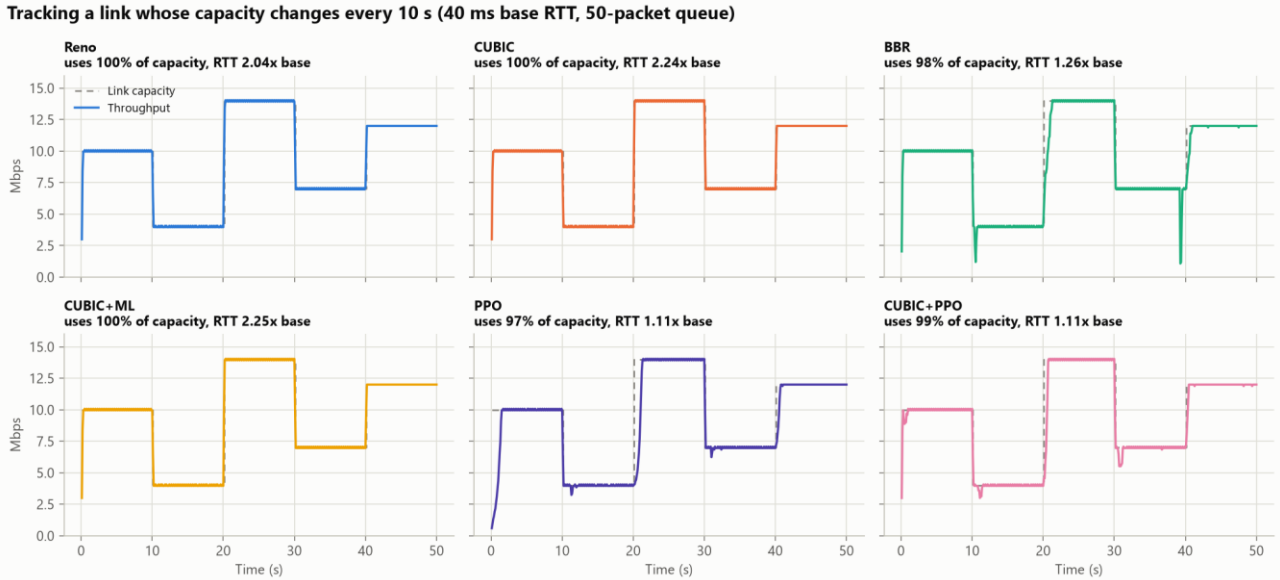
PPO uses 98.8% of capacity at only 1.07× the base RTT. Reno and CUBIC lose throughput on lossy links and still nearly double the delay.

Results: random loss



At 5% loss CUBIC delivers 1.6 Mbps; CUBIC+ML delivers 9.6 and PPO 9.5. PPO was never told which losses are random.

Results: changing capacity



1.2×

worst-case RTT with PPO, relative to the base

4.6×

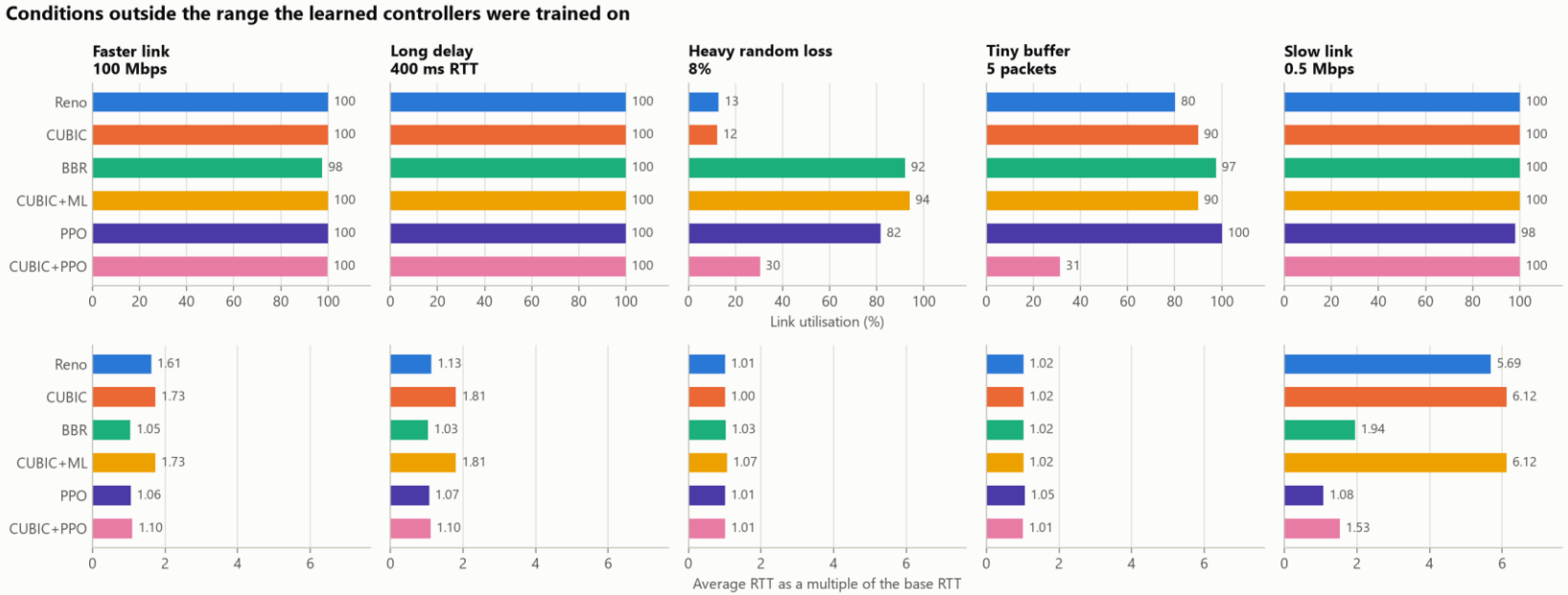
with CUBIC on the same link

97%+

of capacity used by every controller

Capacity steps 10, 4, 14, 7, 12 Mbps. Worst case is the 95th percentile RTT.

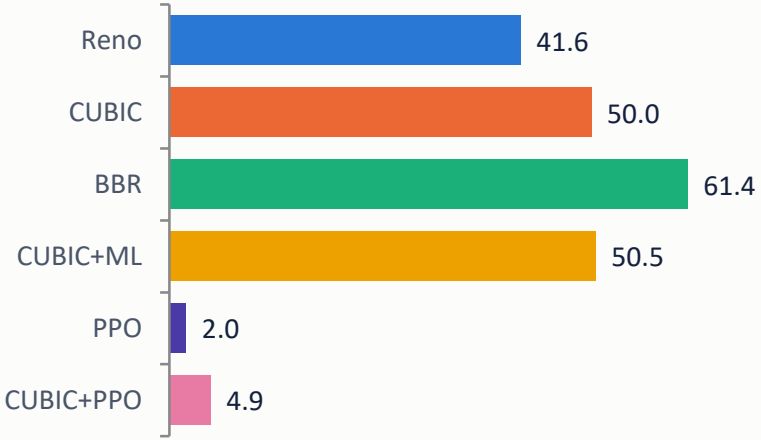
Results: unfamiliar conditions



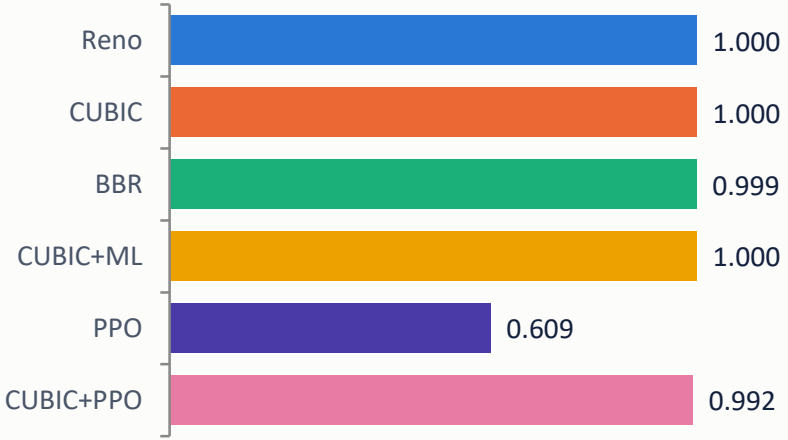
PPO holds at least 98% utilisation in four of five cases. The hybrid fails under 8% loss and with a 5-packet buffer.

Results: fairness

Share of a link beside one CUBIC flow (%)



Jain index, two flows of the same controller



PPO gets 2% beside CUBIC: it reads the delay CUBIC causes as congestion and backs off. CUBIC+ML shares evenly.

STAGE 6 OF 8

Fixing the weaknesses

WHY THIS STAGE

Stage 5 found that the agent gets 2% of a link beside CUBIC, and that it collapses with 2 ms of RTT jitter. Can both be fixed?

Fixing fairness: PPO-Switch

Learned policy (PPO-J)
low delay

⟶ another flow holds the queue

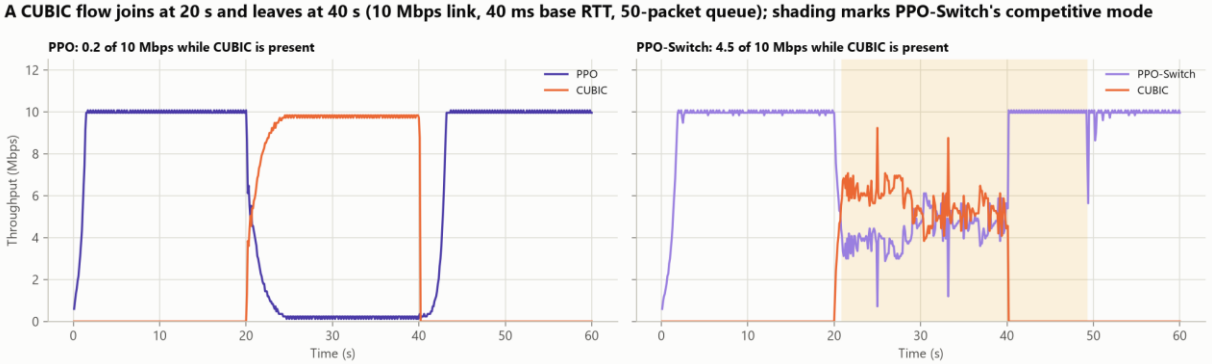
⟵ probe drains the queue: it was ours, switch back

CUBIC+ML
competes fairly

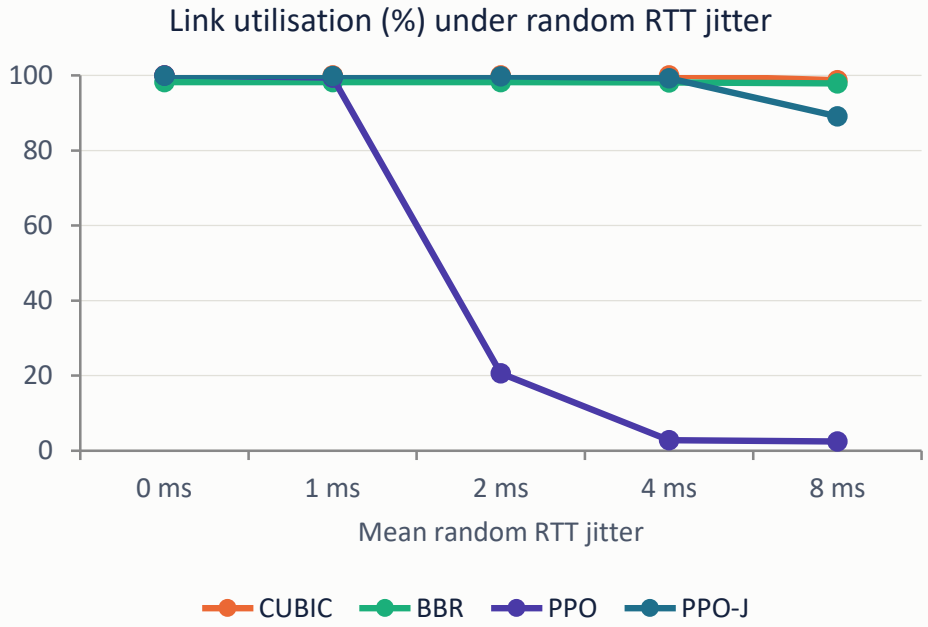
2% → 47%
share beside one CUBIC flow

49% · 43%
beside Reno · beside BBR

0 of 30
mistaken switches when alone



Real networks jitter: PPO-J



3% → 99%

utilisation at 4 ms of jitter, PPO → PPO-J

- Trained with random jitter in 75% of episodes
- Reads the minimum RTT, not the average, as Copa does
- Clean links: 98.5% utilisation, RTT 1.17× base (PPO: 98.8%, 1.07×)

STAGE 7 OF 8

A real Wi-Fi network

WHY THIS STAGE

Both weaknesses are fixed in simulation. Does any of it hold on a real network?

Testing over a real network



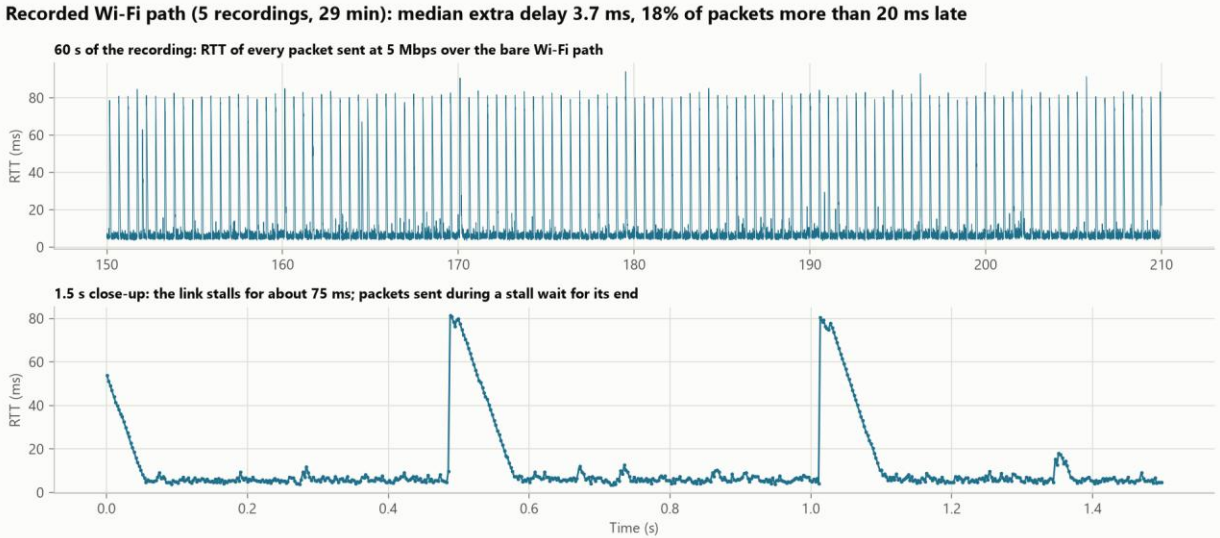
On real Wi-Fi (second run)	Simulator	Real Wi-Fi
CUBIC alone	10.0 Mbps	8.6 Mbps, 70 ms
CUBIC+ML / CUBIC, 1% loss	9.9 / 3.4 Mbps	**8.7 / 2.9 Mbps**
BBR alone	9.9 Mbps	9.4 Mbps, 33% loss
PPO / PPO-J alone	10.0 Mbps	0.3 / 3.3 Mbps
PPO-Switch beside CUBIC	44% of the link	**45% of the link**

CARRIED OVER

The loss classifier and the fairness fix.

Not the learned rate controllers: why, on the next slide.

What real Wi-Fi does



75 ms

stalls, about twice a second

18%

of packets more than 20 ms late

±1.3 Mbps

replay of the recordings vs the real test

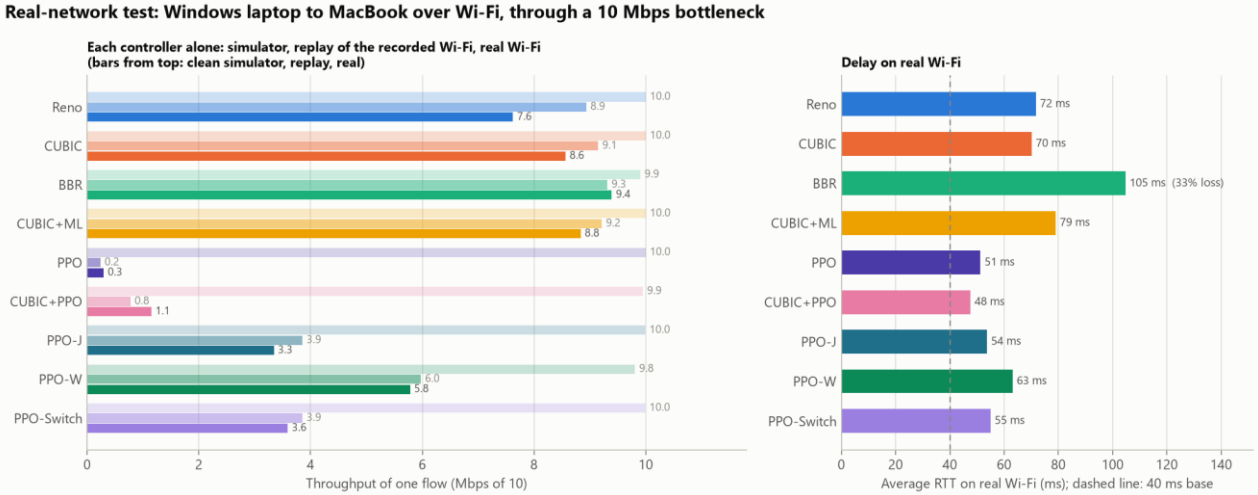
STAGE 8 OF 8

Learning from the real network

WHY THIS STAGE

On real Wi-Fi the agent fell to 0.3 Mbps, because the Wi-Fi stalls for 75 ms twice a second. Can it learn from recordings of that network?

Training on recorded Wi-Fi: PPO-W



0.3 → 5.8

Mbps on real Wi-Fi, PPO → PPO-W

6.0

Mbps predicted by the replay

63 ms

RTT, vs 70 ms for CUBIC

Live dashboard



- Network presets, or set bandwidth, delay, queue size and loss by hand
- Up to four controllers, side by side or competing on one link
- Throughput, RTT, queue and window update while it runs
- Summary table and Jain fairness index after each run

streamlit run dashboard/app.py

CONCLUSION

What each stage achieved

Stage	What we did	What we found	What it led to
1	Simulator; Reno, CUBIC, BBR	Queues kept full; CUBIC 3.7 Mbps at 1% loss	Two weaknesses to fix
2	Loss classifier inside CUBIC	9.6 Mbps at 5% loss (CUBIC 1.6); fair	Random loss fixed
3	LSTM forecast of the RTT	12% better than the last value	A modest gain
4	PPO agent sets the rate	98.8% of capacity at 1.07× base RTT	Full queues fixed, alone
5	Six controllers compared	**Agent gets 2% beside CUBIC**	Fix the agent
6	PPO-Switch; PPO-J	47% beside CUBIC; 99% at 4 ms jitter	Test for real
7	Real Wi-Fi, two laptops	**ML and fairness hold; agent 0.3 Mbps**	Learn from the network
8	PPO-W on 29 min of Wi-Fi	5.8 Mbps real (6.0 predicted)	The conclusions

CONCLUSION

The numbers at a glance

Measure	Reno	CUBIC	BBR	CUBIC+ML	PPO	CUBIC+PPO	PPO-Switch
Utilisation, 30 unseen links (%)	81.6	83.2	98.3	**99.5**	98.8	95.8	98.5
Average RTT ÷ base RTT	1.63	1.77	1.18	1.88	**1.07**	1.15	1.17
Mbps at 5% random loss (of 10)	1.7	1.6	9.5	9.6	9.5	4.4	**9.7**
Share beside one CUBIC flow (%)	42	50	61	**51**	2	5	47
Jain index, two of its own kind	**1.000**	**1.000**	0.999	**1.000**	0.609	0.992	0.645
Utilisation with 4 ms jitter (%)	99.7	**100.0**	98.1	**100.0**	2.8	22.9	99.2
Real Wi-Fi, alone (Mbps)	7.6	8.6	9.4*	8.8	0.3	1.2	3.6

Shaded = best in the row. For sharing, best means closest to an equal 50%. *BBR lost 33% of its packets on real Wi-Fi.

CONCLUSION

Limitations

One real network

The Wi-Fi test covers one path, in user-space UDP; PPO-W is trained on it.

Two learned flows

PPO-Switch fixes sharing with CUBIC; two learned flows still split a link unevenly.

The hybrid is not robust

CUBIC+PPO fails at 8% loss and with very small buffers.

Below CUBIC on Wi-Fi

PPO-W gets 5.8 Mbps against CUBIC's 8.6 there, with less delay.

One run per policy

Small gaps between variants could change with another seed.

LSTM not used for control

Its gain is modest and no controller uses it yet.

CONCLUSION

What we take from this project

1 Small and inside

A learned part inside CUBIC was the most reliable gain, in simulation and on real Wi-Fi.

2 Sharing needs design

More training did not stop starvation beside CUBIC; a competitive mode did.

3 Train for reality

Trained on exact RTTs, the agent failed with 2 ms of jitter.

4 Measure, then replay

29 min of Wi-Fi predicted real results within 1.3 Mbps, and took the agent 0.3 → 5.8 Mbps.

5 Reward what it controls

Counting only its own queue raised the agent from 42% to 60%.

6 Test on the real thing

The simulator ranked the agent first; real Wi-Fi did not.

Learned components can fix both weaknesses; learned rate control still needs real data, the right reward and a mechanism for sharing.

Key references

- Jacobson. Congestion avoidance and control. SIGCOMM 1988
- Mathis et al. Macroscopic behavior of TCP congestion avoidance. CCR 1997
- Ha, Rhee, Xu. CUBIC. ACM SIGOPS OSR 2008
- Winstein, Balakrishnan. TCP ex Machina (Remy). SIGCOMM 2013
- Cardwell et al. BBR. ACM Queue 2016
- Schulman et al. Proximal policy optimization. arXiv 2017
- Arun, Balakrishnan. Copa. NSDI 2018
- Dong et al. PCC Vivace. NSDI 2018
- Jay et al. Aurora: deep RL for congestion control. ICML 2019
- Abbasloo, Yen, Chao. Orca. SIGCOMM 2020
- Yen, Abbasloo, Chao. Sage. SIGCOMM 2023
- Liao et al. Astraea. EuroSys 2024



Thank you

Questions, and a live demo in the dashboard